

Total Members	1
Members	**Name** / **Role** / **Email** Mhiah / Founder / Solo Builder / omariam4122@gmail.com

Overview

Project	RYO Pulse
Track	Track 3 (New Skills - scan_persistence) + Track 2 (Dashboards & Interfaces)
Problem	RYO's scan_market shows which tokens are hot right now, but one appearance isn't a pattern. Agents and traders treat a single hit as a lasting signal, and most scanners read a data outage as a token disappearing.
Solution	RYO Pulse saves scan_market snapshots on a schedule with fixed settings and classifies each token across them as persistent, emerging, transient, absent or insufficient. Failed scans lower coverage but never count as absence. Every answer comes with plain reasons and a replay_hash, so anyone can recompute it from the stored scans and get the same result.
Key Features	scan_persistence skill (MCP-style tool definition, JSON in / JSON out); 5 honest statuses incl. 'insufficient' instead of guessing; outage-proof (failed scans are not absence); only same-settings scans are compared (profile hash); replay_hash + replay command; market_overview regime context (REGIME_CHANGED); keyboard-friendly 'What changed and why' dashboard with editable Settings; retries with backoff + append-only store; 48 offline tests.
Target Users	AI agents and traders that act on RYO scan_market hits and need to know if a token keeps showing up before acting, and judges who want to re-check any verdict from the saved scans.

Scope	A collector for live scan_market + market_overview, an append-only snapshot store, a pure deterministic classifier (the Track 3 skill), a CLI (discover, collect, pulse, replay, board, serve) and a read-only web dashboard (Track 2). Run live against the real RYO MCP endpoint: 114 scans from Sep 27 to Oct 2, 2026.
Limitations	Needs history: says 'insufficient' until it has at least 3 good comparable scans and 60% coverage. The hosted dashboard (ryo-pulse.vercel.app) is a saved copy of the real scans and does not update; live data needs the collector running locally with a RYO key. The tool definition will be aligned with RYO's official skill spec once published.

Tech Stack

Frontend Stack	None (static HTML/CSS/vanilla JS served by the Python app, no framework or build step; bundled DejaVu Sans font)
Backend Stack	Python 3.11+, pydantic (models), httpx (MCP client), standard-library HTTP server for the dashboard; Vercel Python function for the hosted copy.
AI Model(s)	None at runtime by design: classification is pure and deterministic, which is what makes replay exact. Claude (Anthropic) was used as a coding assistant during the build. Demo video narration uses the open Kokoro-82M text-to-speech model.
Other Technologies	rich (CLI output), pytest (48 tests), Vercel (hosting the dashboard), ffmpeg (demo video)

Repository / Demo

Github Repository	https://github.com/Mhiah/RYO-PULSE (mirrored to the organizers' repo RYO-Digital/ryochan-hackathon_repository-294)
Demo Video	https://1drv.ms/v/c/0328d6730c05f4c7/IQD9U4l6f6l1RI4HrTio51GSAcHKinmWAF82OwUjiJ5rw2U?e=sHCzjh
Documentation	README.md (repo root); design and skill contract in docs/design.md; JSON Schemas and the scan_persistence tool definition in docs/schema/; hosted dashboard at https://ryo-pulse.vercel.app

Testing Information

How to run the Project	CLI tool (ryo-pulse) plus pytest test suite. Quickest look: the hosted dashboard https://ryo-pulse.vercel.app, or run offline on the labelled fixture (commands below).
Prerequisites	Python 3.11+; pip. A RYO MCP key only for live data.
Installation Steps	1) Clone the repo 2) python -m venv .venv and activate it 3) pip install -e ".[dev]" 4) For live data: copy .env.example to .env and fill in RYO_MCP_URL and RYO_MCP_KEY
Environment Variables	RYO_MCP_URL, RYO_MCP_KEY (RYO MCP); optional: RYO_AUTH_HEADER, RYO_AUTH_SCHEME, RYO_SCAN_TOKENS_PATH, RYO_SCAN_STATUS_PATH, RYO_SCAN_FAIL_STATUSES, RYO_OVERVIEW_REGIME_PATH, PULSE_DATA_DIR. All listed in .env.example.
Build Command	None
Run Command	Offline: ryo-pulse serve --fixture tests/fixtures/demo_window.json | Live: ryo-pulse discover -p top_n=20; ryo-pulse collect -p top_n=20 --every 30; ryo-pulse serve; ryo-pulse pulse OMI --save result.json; ryo-pulse replay result.json
Test Command	pytest
Test Account(s)	None required. RYO tools are read-only research APIs authenticated by an environment-variable key, not a login.